11. Generics: Type Parameters and Constraints

11. Generics: Type Parameters and Constraints

Phase: Phase 2 — Core language depth
Estimated time: 1–2 days
Slug: `11-generics`

Learning objectives

- Write generic functions with type parameters
- Use constraints from constraints package / comparable
- Know when generics help vs when interfaces suffice
- Avoid over-generic APIs

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Type parameters

```
func Keys[K comparable, V any](m map[K]V) []K {
    out := make([]K, 0, len(m))
    for k := range m {
        out = append(out, k)
    }
    return out
}
```

Constraints

```
type Number interface {
    ~int | ~int64 | ~float64
}

func Sum[T Number](xs []T) T {
    var s T
    for _, v := range xs {
        s += v
    }
    return s
}
```

`comparable` is required for map keys and ==.

When not to use generics

If a small interface expresses the behavior better, prefer interfaces. Generics shine for containers, algorithms, and removing `any` casts.

Common mistakes (especially from Python instincts)

- Genericizing every function “just in case”
- Forgetting `comparable` for map keys
- Fighting the type checker instead of simplifying the API

Exercises

Work in `exercises/11-generics/`. Read that folder’s `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/11-generics/` until you have an honest failing attempt.

Self-check

1. What does `comparable` allow?
2. When prefer interface methods over type parameters?
3. What does `~int` mean in a constraint?

Further reading

- https://go.dev/doc/tutorial/generics
- https://go.dev/blog/intro-generics